Proofs, Computability, and Complexity

Week 1: Algorithmic Reasoning and First-Order Logic (Warm-Up)

https://brunoloff.wordpress.com/proofs-computability-and-complexity-2025/

Context These are **review exercises**! If you find them too easy, that's okay. If you find them too difficult, you need to practice them more. For that purpose, there are many more exercises than the ones we will solve in class. Try to solve them on your own, then compare your solution with the answer. Once you are satisfied you can solve all exercises of the same kind, solve the corresponding homework exercise.

Goals Practice precise algorithmic thinking (inputs/outputs, steps, conditions, loops, invariants) and basic logic (translation, negation, quantifiers, truth, and simple entailment).

Instructions Unless stated otherwise, the domain of discourse for first-order logic is $\mathbb{N}$. You may answer in clear English (not Portuguese!), pseudocode, and symbolic logic as requested. Show your work.

Homework and Grading Every week there will be a homework assignment, testing whether you understood the exercises in the exercise sheet. You turn in your homework at the start of the next Exercise session. Your homeworks will be graded by Francesca, and she will give you feedback on the solution you submitted.

Your homework score will count for the final grade *multiplicatively*, as follows:

- When we give score to a test, we will look at the average score of all the homeworks that cover the test's material.
- If your scored between 10 and 14.9 (out of 20) on your homeworks, you will get a 10% bonus on your test grade.
- Between 15.0 and 20, you will get a 20% bonus (obviously to a maximum of 20).

The idea is to encourage you to do the homework, while defeating the purpose of turning in homework you didn't solve yourself (if you copy homework, you still won't know how to answer questions in the test, and 120% of zero is still zero).

Section A. Algorithmic reasoning

> Bruno: I have found that many FCUL math students are missing basic agility in algorithmic thinking. This is not OK in the 21st century, so we hope to correct for this in this course, at least to some degree. Here are a few elementary exercises to train this kind of thinking.

1. **Unambiguous instructions.** Describe the method you have learned in primary school to sum two numbers, by a step-by-step list of instructions. The instructions should be very clear, complete and precise, like a recipe. Someone who does not know how to sum should be able to follow the instructions and sum two numbers (e.g. by listening to the instructions, assuming they also don't know how to read). Try to follow the instructions yourself, as if you didn't know how to sum, and see if they work.

2. Dry run and "what does it do?"

```
function mystery(n):  # n is a nonnegative integer in base 10
    s = 0
    while n > 0:
        s = s + (n % 10) # a % b is the remainder of the division of a by b
        n = n // 10      # a // b is the quotient of the division of a by b
    return s
```

(**a**) Compute `mystery(2025)`, `mystery(999)`, `mystery(0)`.

(**b**) In one sentence: what function of n does `mystery` compute?

Solution. (a) $2025 \mapsto 2+0+2+5 = 9$; $999 \mapsto 27$; $0 \mapsto 0$. (b) The sum of the base-10 digits of n.

3. Euclid by hand (termination and trace).

Recall is Euclid's algorithm for computing the GCD of two numbers.

```
function gcd(a, b):
    if b = 0:
        return a
    else:
        return gcd(b, a mod b)
```

(**a**) Run Euclid's algorithm on $(252, 198)$. List the sequence of remainders and state the gcd.

(**b**) Briefly explain why this algorithm must terminate on all pairs of nonnegative integers.

(**c**) *Show that the algorithm runs in time $O(\log(\max(a, b)))$

Solution. $252 = 1 \cdot 198 + 54$, $198 = 3 \cdot 54 + 36$, $54 = 1 \cdot 36 + 18$, $36 = 2 \cdot 18 + 0$. Hence $\gcd(252, 198) = 18$. Termination: the nonnegative remainders strictly decrease; a strictly decreasing sequence in $\mathbb{N}$ is finite.

(Complexity) Let $M_0 = \max\{a, b\}$. Each step replaces (a, b) by $(b, a \bmod b)$ with $a \bmod b < b$. If $b \le a/2$ then after one step the new maximum is $\le a/2$. If $b > a/2$ then $a \bmod b = a - b < a/2$, and after the next step both arguments are $< a/2$. Thus every one or two iterations reduce the current maximum by a factor of at least 2, so the number of iterations is $\le 2\lfloor \log_2 M_0 \rfloor + 1 = O(\log M_0)$.

4. Loop invariant (maximum of a list).

```
function max_of_list(A):  # A is a nonempty list of integers
    m = A[0]
    for i from 1 to len(A)-1:
        if A[i] > m:
            m = A[i]
    return m
```

(**a**) State a loop invariant which is always true at the start of every iteration i. (An invariant is a property of the program state which always holds at a given point).

(**b**) Use your loop invariant to justify correctness of the algorithm in 3–5 sentences.

Solution. The invariant is: at the start of the loop $m = \max\{A[0], \ldots, A[i-1]\}$.

It holds at the first iteration: $i = 1$, $m = A[0]$ is the max of the prefix.

It is preserved at each loop cycle: comparing $A[i]$ to m updates m iff $A[i]$ is larger, preserving the invariant.

Correctness: when the loop ends, $i = |A|$, so m is the maximum of the entire list.

5. **Small-n cost counting.** Using the same ("python-like") pseudocode notation as in the `mystery` and `max_of_lists` algorithms above, write an algorithm that decides if a list A of length n has duplicates by checking all pairs $i < j$ and comparing $A[i]$ to $A[j]$.

   Give a formula in n for the worst-case number of comparisons.

   Can you imagine a faster algorithm?

   Solution. Algorithm (checks all pairs $i < j$):

```
function has_duplicates(A):  # A is a list of length n
    n = len(A)
    for i from 0 to n-2:
        for j from i+1 to n-1:
            if A[i] == A[j]:
                return True
    return False
```

   The formula is: $\binom{n}{2} = \frac{n(n-1)}{2}$.

   A faster algorithm could be obtained, e.g., by first sorting the list, and then traversing it once to find duplicates. Such a strategy can be made to run in $O(n \log n)$ steps using, e.g., heap sort.

6. **Histogram of digits.** Write pseudocode for the following task. Input: a list A of n integers, each between 0 and 9. Output: a list C of length 10 where $C[d]$ counts the number of times that digit d appears in A.

   (**a**) Write clear pseudocode.

   (**b**) Run it on $A = [3, 1, 4, 1, 5, 9, 2, 6, 5]$.

   (**c**) How many assignments to entries of C occur as a function of n?

   Solution. (a)

```
function digit_histogram(A):
    C = array(10) filled with 0
    for each x in A:
        C[x] = C[x] + 1
    return C
```

   (b) $C = [0, 2, 1, 1, 1, 2, 1, 0, 0, 1]$ (indices 0..9). (c) Exactly n assignments to entries of C.

7. In-place reversal with invariants and tight cost. You are given an array A of length $n \geq 0$. Design an algorithm that reverses A in place using two indices (no auxiliary array of length n).

(**a**) Write clear pseudocode using two indices ℓ and r.

(**b**) State a loop invariant that is true at the start of every iteration and suffices to prove correctness.

(**c**) Prove termination by exhibiting a variant (a nonnegative integer measure that strictly decreases each iteration).

(**d**) Derive a tight formula for the exact number of swaps and for the exact number of element *assignments* performed by your algorithm as a function of n (assume a swap of two distinct positions is implemented as three assignments).

(**e**) Prove a lower bound: any algorithm that only uses swaps of two array positions to reverse A must perform at least $\lfloor n/2 \rfloor$ swaps in the worst case.

Solution. (a) Pseudocode:

```
function reverse_in_place(A):
    n = len(A)
    l, r = 0, n-1
    while l < r:
        swap(A[l], A[r])    # 3 assignments
        l = l + 1
        r = r - 1
    return A
```

(b) Invariant (at loop start): For all $i < \ell$, $A[i] = A_0[n-1-i]$, and for all $j > r$, $A[j] = A_0[n-1-j]$, where A_0 is the initial array. Also $\ell \leq r + 1$. So the outer segments are already reversed and in their final positions; the middle segment $A[\ell..r]$ is not yet processed.

(c) Variant: $V = r - \ell + 1$ (the size of the unprocessed middle). Each iteration increments ℓ and decrements r, so V decreases by 2 when $\ell < r$. Since $V \in \mathbb{N}$ and strictly decreases, termination follows.

(d) Costs: The loop executes exactly $\lfloor n/2 \rfloor$ iterations (pairs $(0, n-1), (1, n-2), \dots$). Thus: - Swaps: exactly $\lfloor n/2 \rfloor$. - Assignments: exactly $3\lfloor n/2 \rfloor$ (by the given swap model).

(e) Lower bound: Consider the reverse permutation ρ on indices $0, \dots, n-1$, given by $\rho(i) = n-1-i$. It decomposes into $\lfloor n/2 \rfloor$ disjoint transpositions $(0\ n\text{–}1)(1\ n\text{–}2)\cdots$. Any algorithm that transforms A to its reversal using only swaps of array positions must realize the permutation ρ on indices, and the minimal number of transpositions needed to realize a permutation equals $n - c$, where c is the number of cycles in its disjoint-cycle decomposition; here $c = \lceil n/2 \rceil$, hence the minimum is $\lfloor n/2 \rfloor$. Therefore no such algorithm can use fewer than $\lfloor n/2 \rfloor$ swaps in the worst case, and the algorithm above is optimal.

8. Base-b conversion, correctness, and complexity. Input: an integer $n \geq 0$ and a base b with $2 \leq b \leq 10$. Output: the base-b representation of n as a list of digits $\langle d_k, \dots, d_1, d_0 \rangle$ in *most-significant-digit first* order, with no leading zeros unless $n = 0$.

(**a**) Write pseudocode that produces the digits in MSD-first order. (You may use a stack or recursion, but not an $O(n)$-length auxiliary array when n is given in unary; aim for $O(\log_b n)$ extra space.)

(**b**) State and prove a loop (or recursion) invariant that relates the partial output and the remaining value of n (e.g., an equation of the form $n = D \cdot b^t + r$ where D encodes the digits already emitted).

(**c**) Prove there are exactly $\lfloor \log_b(n) \rfloor + 1$ digits when $n > 0$, and bound the number of divisions and remainders your algorithm performs to within a constant factor of this value.

(**d**) Prove uniqueness: if two MSD-first digit lists over $\{0, 1, \ldots, b-1\}$ represent the same n with no leading zeros (unless $n = 0$), then the lists are identical.

Solution. (a) Use recursion to emit MSD-first:

```
procedure emit_base_b(n, b):   # prints digits MSD-first
    if n < b:
        output n
    else:
        emit_base_b(n // b, b)
        output (n % b)
```

This uses $O(\log_b n)$ stack depth and $O(1)$ extra work per frame.

(b) Correctness invariant (induction on recursion depth): For a call with argument $n > 0$, before the final `output (n % b)` executes, the recursive call has already printed the base-b expansion of $\lfloor n/b \rfloor$ in MSD-first order. Appending $n \bmod b$ therefore prints the expansion of $n = \lfloor n/b \rfloor \cdot b + (n \bmod b)$ in MSD-first order. Base case $n < b$ prints a single digit and is correct.

(c) Number of digits: For $n > 0$, let $k = \lfloor \log_b n \rfloor$. Then $b^k \le n < b^{k+1}$, so the expansion has exactly $k + 1$ digits. The procedure performs one division and one remainder per non-base case frame, i.e. exactly k of each when $n > 0$. Thus the total number of such operations is $\Theta(\lfloor \log_b n \rfloor + 1)$.

(d) Uniqueness: Suppose $n > 0$ has two expansions without leading zeros, $n = \sum_{i=0}^{k} d_i b^i = \sum_{i=0}^{\ell} e_i b^i$, with $d_k \neq 0$, $e_\ell \neq 0$. If $k \neq \ell$, then w.l.o.g. $k > \ell$ gives $n \ge b^k > \sum_{i=0}^{k-1} (b-1) b^i = b^k - 1$, contradiction. Hence $k = \ell$. Subtracting, $\sum_{i=0}^{k} (d_i - e_i) b^i = 0$. Let j be the largest index with $d_j \neq e_j$; then $0 = \sum_{i=0}^{j} (d_i - e_i) b^i \equiv (d_j - e_j) b^j \pmod{b^{j+1}}$, which implies $d_j = e_j$, contradiction. Therefore the digits are identical.

Section B. Propositional logic

Bruno: You should know your basic *true and falses*. Clear understanding of basic propositional logic was for many centuries considered a delicate and refined art. Nowadays it is considered routine, and it should be routine for you.

1. Truth table and tautology. Build a truth table for the formula

$$\big(P \to Q\big) \ \wedge \ \big(Q \to R\big) \ \to \ \big(P \to R\big).$$

Is it a tautology? Justify from the table.

Solution. Yes, it is a tautology (hypothetical syllogism). In every assignment where $P \to Q$ and $Q \to R$ are true, P being true forces Q true and then R true, so $P \to R$ is true; if P is false then $P \to R$ is vacuously true.

2. **Simplifying a conjunction.** Translate "If it is raining, then the ground is wet; and if the ground is not wet, then it is not raining." using R for "raining" and W for "ground is wet". Then simplify logically as far as you can.

   Solution. $(R \to W) \ \wedge \ (\neg W \to \neg R)$. The second conjunct is the contrapositive of the first, so the conjunction is equivalent to just $R \to W$.

3. **Currying/uncurrying and distribution without truth tables.** Using only the basic equivalences — associativity, commutativity, distributivity of $\vee$ and $\wedge$, De Morgan, and perhaps rewriting implication as $\neg P \vee Q$ — prove the following:

   (a) $(P \wedge Q) \to R \ \leftrightarrow \ P \to (Q \to R)$, (b) $(P \vee Q) \to R \ \leftrightarrow \ (P \to R) \wedge (Q \to R)$.

   Give step-by-step transformations for each equivalence (no truth tables).

   Solution. (a) $(P \wedge Q) \to R \equiv \neg(P \wedge Q) \vee R \equiv (\neg P \vee \neg Q) \vee R \equiv \neg P \vee (\neg Q \vee R) \equiv P \to (\neg Q \vee R) \equiv P \to (Q \to R)$.
   (b) $(P \vee Q) \to R \equiv \neg(P \vee Q) \vee R \equiv (\neg P \wedge \neg Q) \vee R \equiv (R \vee \neg P) \wedge (R \vee \neg Q) \equiv (P \to R) \wedge (Q \to R)$.

4. **Functional completeness of $\{\to, \bot\}$.** Let $\bot$ denote a constant false formula.

   (**a**) Define $\neg P$, $P \vee Q$, $P \wedge Q$, and $P \leftrightarrow Q$ using only $\to$ and $\bot$. Justify each definition via equivalences.

   (**b**) Using your primitives, express exclusive-or $P \oplus Q$ (true exactly when P and Q have different truth values) with only $\to$ and $\bot$.

   Solution. (a) Definitions and justifications:

$$\neg P \ \equiv \ P \to \bot \quad \text{(by definition of implication).}$$

$$P \vee Q \ \equiv \ (P \to \bot) \to Q.$$

$$P \wedge Q \ \equiv \ \neg(P \to \neg Q) \ \equiv \ (P \to (Q \to \bot)) \to \bot.$$

$$P \leftrightarrow Q \ \equiv \ (P \to Q) \wedge (Q \to P) \ \equiv \ \Big((P \to Q) \ \to \ \big((Q \to P) \to \bot\big)\Big) \ \to \ \bot.$$

   (b) Start with the disjunctive form:

$$P \oplus Q \ \equiv \ (P \wedge \neg Q) \ \vee \ (\neg P \wedge Q).$$

   Expanding each connective using only $\to$ and $\bot$ yields

$$P \oplus Q \ \equiv \ \Big(\big((P \to ((Q \to \bot) \to \bot)) \to \bot\big) \to \bot\Big) \ \to \ \Big(\big((P \to \bot) \to (Q \to \bot)\big) \to \bot\Big).$$

   This uses only $\to$ and $\bot$.

Section C. Expressing definitions and statements using First-order logic (FOL)

Bruno: It is a highly non-trivial fact that essentially any mathematical statement can be written in first order logic (with some, not very important, caveats). This section is meant to illustrate this process. The invention of FOL is owed to Frege, who died in 1925, so this year we celebrate the 100th aniversary of his death.

1. **Translate to FOL.** Write each as a formula over $\mathbb{N} = \{0, 1, \ldots\}$. Use only symbols $0, 1, +, \times, <, =$ on $\mathbb{N}$; predicates $\text{Even}(x)$, $\text{Prime}(x)$, $\text{Divides}(x, y)$, and uniqueness $\exists! x\, \varphi(x)$ are allowed.

   (**a**) "Every even natural number is divisible by 2."

   (**b**) "There is a prime that is even."

   (**c**) "Every natural number has a larger prime."

   (**d**) "There exists exactly one integer whose square is 0."

   (**e**) Now write a uniqueness quantifier $\exists! x\, \varphi(x)$ using only $\forall$ and $\exists$.

   Solution. (a) $\forall x\, (\text{Even}(x) \to \text{Divides}(2, x))$.
   (b) $\exists x\, (\text{Prime}(x) \wedge \text{Even}(x))$.
   (c) $\forall x\, \exists y\, (y > x \wedge \text{Prime}(y))$.
   (d) $\exists! x\, (x \times x = 0)$.
   (e) $\exists x\ \varphi(x) \wedge \forall y (\varphi(y) \to x = y)$.

2. **Define basic number-theoretic predicates from $\{0, 1, +, \times, =\}$.** Above you used predicates such as $\text{Prime}(x)$. You should realize, of course, that these can be expressed from more primite concepts, such as sum and multiplication.

   Work over the domain $\mathbb{N}$. Use only terms built from $0, 1, +, \times$, and the relation symbol $=$. (Numerals $2, 3, \ldots$ are shorthands for repeated $+1$, e.g. $2 := 1 + 1$, $3 := 1 + 1 + 1$). Starting from such simple language, build up each of the following definitions using formulas of first order logic. (Once you've defined a concept, you can use it in the future.)

   (**a**) Not equal: $x \neq y$.

   Solution. $x \neq y :\ \neg(x = y)$.

   (**b**) Greater than: $x > y$.

   Solution. $x > y\ :\ \exists z\ z \neq 0 \wedge x + z = y$

   (**c**) Even / Odd: $\text{Even}(x)$ and $\text{Odd}(x)$.

   Solution. $\text{Even}(x)\ :\ \exists y \left(x = 2 \times y\right)$. $\quad \text{Odd}(x)\ :\ \exists y \left(x = 2 \times y + 1\right)$.

   (**d**) Divisibility: $\text{Divides}(a, b)$ meaning "a divides b".

   Solution. $\text{Divides}(a, b)\ :\ \exists k \left(b = a \times k\right)$.

(**e**) Congruence modulo m: $x \equiv y \pmod{m}$.

Solution. $x \equiv y \pmod{m} \;:\; m > 0 \;\wedge\; \exists k \big(x = y + m \times k\big)$.

(**f**) Being prime (standard convention: primes are > 1). Give two equivalent definitions.

Solution. (i) Using only $+, \times$ via factorisations:

$$\text{Prime}(p) \;:\; p > 1 \;\wedge\; \forall a \forall b \Big((a > 0 \wedge b > 0 \wedge p = a \times b) \;\rightarrow\; (a = 1 \;\vee\; b = 1)\Big).$$

(ii) Using Divides:

$$\text{Prime}(p) \;:\; p > 1 \;\wedge\; \forall a \Big((a > 1 \wedge \text{Divides}(a, p)) \;\rightarrow\; a = p\Big).$$

(**g**) Being Composite: Composite(n).

Solution. $\text{Composite}(n) \;:\; n > 1 \;\wedge\; \exists a \exists b \big(a > 1 \wedge b > 1 \wedge n = a \times b\big)$.

(**h**) Being Coprime: Coprime(a, b).

Solution. One could use Bezout's form: $\text{Coprime}(a, b) \;:\; \exists x \exists y \big(ax + by = 1\big)$.

(**i**) Being a perfect square: Square(n).

Solution. $\text{Square}(n) \;:\; \exists t \big(n = t \times t\big)$.

(**j**) Much less obvious is how to define exponentiation $x = y^z$. One way to do it is using Godel's β predicate, which allows us to encode any element of $\mathbb{N}^*$ (finite sequences of natural numbers) with two elements $\mathbb{N}^2$:

$$\beta(b, c, i, r) \;:\; b = r \mod (i + 1) \cdot c + 1.$$

One must prove that for every $(a_1, \ldots, a_k)$ there exists some b, c such that $\beta(b, c, i, x)$ holds (for $1 \leq i \leq k$) iff $r = a_i$. You can think why, as an exercise (hint: Chinese). Now show how to encode the predicate $x = y^n$ using Godel's β.

Solution. $x = y^n \;:\; \exists b, c\, \beta(b, c, 1, 1) \wedge \beta(b, c, n + 1, x) \wedge$
$\forall i \leq n\, \exists z\, \beta(b, c, i, z) \wedge \beta(b, c, i + 1, yz)$.

3. Translate significant arithmetic statements into first-order logic over $\mathbb{Z}$. Use only the base language $\{0, 1, +, \times, <\}$ together with the predicates you defined above (e.g. Prime, Divides, Coprime). Express "infinitely many" as "for every bound N there exists an object $> N$ with the property".

(**a**) There are infinitely many primes.

Solution. $\forall N \exists p \big(p > N \;\wedge\; \text{Prime}(p)\big)$.

(**b**) Bertrand's postulate (proven by Chebyshev): for every $n > 1$ there exists a prime p with $n < p < 2n$.

Solution. $\forall n \Big(n > 1 \;\rightarrow\; \exists p \big(\text{Prime}(p) \;\wedge\; n < p \;\wedge\; p < n + n\big)\Big)$.

(**c**) Dirichlet (state form): for all a, m with $m > 0$ and $\text{Coprime}(a, m)$ there are infinitely many primes $p \equiv a \pmod{m}$.

Solution. $\forall a\, \forall m \Big((m > 0 \wedge \text{Coprime}(a, m)) \;\to\; \forall N\, \exists p\, \exists k \, \big(p > N \;\wedge\; \text{Prime}(p) \;\wedge\; p = a + km\big)\Big)$.

(**d**) Euclid's lemma: if a prime p divides ab, then p divides a or p divides b.

Solution. $\forall p\, \forall a\, \forall b \, \Big(\text{Prime}(p) \wedge \text{Divides}\big(p, a \times b\big) \;\to\; \text{Divides}(p, a) \;\vee\; \text{Divides}(p, b)\Big)$.

(**e**) Every integer $n > 1$ has a prime divisor.

Solution. $\forall n \, \Big(n > 1 \;\to\; \exists p \, \big(\text{Prime}(p) \;\wedge\; \text{Divides}(p, n)\big)\Big)$.

(**f**) Goldbach's conjecture: every even integer $n > 2$ is a sum of two primes.

Solution. $\forall n \, \Big((n > 2 \wedge \text{Even}(n)) \;\to\; \exists p\, \exists q \, \big(\text{Prime}(p) \wedge \text{Prime}(q) \wedge n = p + q\big)\Big)$.

(**g**) The Twin primes conjecture: there are infinitely many primes p with $p + 2$ also prime.

Solution. $\forall N\, \exists p \, \big(p > N \;\wedge\; \text{Prime}(p) \;\wedge\; \text{Prime}(p + (1 + 1))\big)$.

(**h**) Fermat's Last Theorem: for every integer $n \geq 3$ there are no positive integers x, y, z with $x^n + y^n = z^n$.

Solution. $\forall n \, \Big(n \geq 3 \;\to\; \neg \exists x\, \exists y\, \exists z\, \exists u\, \exists v\, \exists w \, \big(x > 0 \wedge y > 0 \wedge z > 0 \wedge u = x^{\,n} \wedge v = y^{\,n} \wedge w = z^{\,n} \wedge u + v = w\big)\Big)$.

Section D. Working with FOL formulas

Bruno: You should have as much agility with FOL as you have with basic propositional logic. In the end, it all works more or less the same because a forall quantifier is just like an AND of infinitely many statements, and an exists quantifier is just an OR of infinitely many statements.

1. **Negate carefully (push negations inward).** Negate your answers to exercise A of the previous section, and push all $\neg$ immediately in front of atomic formulas.

   Solution. (a) $\exists x \, (\text{Even}(x) \wedge \neg \, \text{Divides}(2, x))$.
   (b) $\forall x \, (\neg \, \text{Prime}(x) \;\vee\; \neg \, \text{Even}(x))$ (equivalently $\forall x \, (\text{Prime}(x) \to \neg \, \text{Even}(x))$).
   (c) $\exists x\, \forall y \, \big(y \leq x \;\vee\; \neg \, \text{Prime}(y)\big)$, since $\neg(y > x \wedge \text{Prime}(y)) \equiv (y \leq x \vee \neg \, \text{Prime}(y))$.
   (d) $\forall x \, (x \times x \neq 0) \;\;\vee\;\; \exists x \exists y \, \big(x \times x = 0 \;\wedge\; y \times y = 0 \;\wedge\; x \neq y\big)$.

2. **Free vs bound variables.** For each formula, list its free variables (if any):

   (**a**) $\forall x \, (\text{Even}(x) \to \exists y \, (x = 2y))$

   (**b**) $\exists y\, \forall x \, (x < y \to \text{Prime}(y))$

   (**c**) $\text{Divides}(x, y) \;\wedge\; \forall y \, (y < 0 \to \text{Even}(y))$

Solution. (a) None. (b) None. (c) x and y are free from the left conjunct; the y in the right conjunct is bound (shadowed).

3. **Truth in a structure.** Over $\mathbb{N}$ with the usual meanings, state whether each is true or false and justify briefly:

   (**a**) $\forall x\, \exists y\, (x < y \wedge \text{Prime}(y))$

   (**b**) $\forall x\, (x > 1 \rightarrow \exists p\, (\text{Prime}(p) \wedge \text{Divides}(p, x)))$

   (**c**) $\exists x\, \forall y\, \text{Divides}(x, y)$

   (**d**) $\forall x\, \exists y\, (x \times y = 1)$

   (**e**) $\forall N\, \exists m\, (m > N \wedge \neg\, \text{Prime}(m))$

   Solution. (a) True: there are infinitely many primes, so for any x there is a prime $> x$.
   (b) True: every $x > 1$ has a prime divisor (factor x; if composite, factor a nontrivial divisor and iterate until a prime is reached).
   (c) True: $x = 1$ divides every natural number.
   (d) False in $\mathbb{N}$: only $x = 1$ has a natural multiplicative inverse; the quantifier is "for all x".
   (e) True: given N, take $m = (N + 1)(N + 2) > N$, which is composite and hence not prime.

4. **Simple entailment with quantifiers.** From premises $\forall x\, (P(x) \rightarrow Q(x))$ and $\forall x\, P(x)$, conclude $\forall x\, Q(x)$. Give a short formal derivation or a clear English proof (3–5 sentences).

   Solution. Let a be arbitrary. From $\forall x\, P(x)$ we get $P(a)$. From $\forall x\, (P(x) \rightarrow Q(x))$ we get $P(a) \rightarrow Q(a)$. By modus ponens, $Q(a)$. Since a was arbitrary, $\forall x\, Q(x)$.

5. **Prenex and negation normal forms with careful renaming.** Convert the formula

$$\neg\, \forall x \Big(P(x) \;\rightarrow\; \exists y \, \big(Q(x, y) \;\wedge\; \forall x \, \big(R(y, x) \;\rightarrow\; \exists z \, S(x, z) \big) \big) \Big)$$

   into both negation normal form (NNF) and then into an equivalent prenex normal form (PNF) with all quantifiers pulled in front. Carefully rename bound variables to avoid capture, and show all equivalence steps you use. The matrix (quantifier-free part) of your PNF should use only $\wedge, \vee, \neg$ applied directly to atomic formulas.

   Recall. A first-order logic formula is in *Negation normal form (NNF)* if it is built from atomic formulas using only the connectives $\wedge$, $\vee$, and $\neg$, with every occurrence of $\neg$ applied directly to an atomic formula. Quantifiers $\forall, \exists$ may appear, but no negation has a quantifier in its scope.

   A first-order logic formula is in *Prenex normal form (PNF)* if it has the shape

$$Q_1 x_1\, Q_2 x_2 \cdots Q_n x_n \;.\; \psi,$$

   where each $Q_i \in \{\forall, \exists\}$ and the matrix ψ is quantifier-free (contains no $\forall$ or $\exists$). No quantifiers occur inside ψ.

   Solution. First, $\alpha = \neg\, \forall x \big(P(x) \rightarrow \exists y (Q(x, y) \wedge \forall x (R(y, x) \rightarrow \exists z\, S(x, z))) \big)$.

   Rename the inner $\forall x$ to $\forall u$ to avoid capture:

$$\alpha \;\equiv\; \neg\, \forall x \Big(P(x) \rightarrow \exists y \big(Q(x, y) \wedge \forall u\, (R(y, u) \rightarrow \exists z\, S(u, z)) \big) \Big).$$

Push $\neg$ inward:

$$\begin{aligned}
&\equiv\ \exists x\,\neg\Big(P(x)\to\exists y\big(Q(x,y)\wedge\forall u\,(R(y,u)\to\exists z\,S(u,z))\big)\Big)\\
&\equiv\ \exists x\,\Big(P(x)\ \wedge\ \neg\exists y\big(Q(x,y)\wedge\forall u\,(R(y,u)\to\exists z\,S(u,z))\big)\Big)\\
&\equiv\ \exists x\,\Big(P(x)\ \wedge\ \forall y\,\neg\big(Q(x,y)\wedge\forall u\,(R(y,u)\to\exists z\,S(u,z))\big)\Big)\\
&\equiv\ \exists x\,\Big(P(x)\ \wedge\ \forall y\,\big(\neg Q(x,y)\ \vee\ \neg\forall u\,(R(y,u)\to\exists z\,S(u,z))\big)\Big)\\
&\equiv\ \exists x\,\Big(P(x)\ \wedge\ \forall y\,\big(\neg Q(x,y)\ \vee\ \exists u\,\neg(R(y,u)\to\exists z\,S(u,z))\big)\Big)\\
&\equiv\ \exists x\,\Big(P(x)\ \wedge\ \forall y\,\big(\neg Q(x,y)\ \vee\ \exists u\,\big(R(y,u)\ \wedge\ \neg\exists z\,S(u,z)\big)\big)\Big)\\
&\equiv\ \exists x\,\Big(P(x)\ \wedge\ \forall y\,\big(\neg Q(x,y)\ \vee\ \exists u\,\big(R(y,u)\ \wedge\ \forall z\,\neg S(u,z)\big)\big)\Big).
\end{aligned}$$

This is NNF (all negations are directly on atoms). For PNF, assume a nonempty domain and pull quantifiers forward, renaming as needed and using, e.g., $\forall y(\phi\vee\exists u\,\psi)\ \equiv\ \forall y\exists u(\phi\vee\psi)$ when u is not free in ϕ, and $\exists u\,(C\wedge\forall z\,D(z))\ \equiv\ \exists u\forall z\,(C\wedge D(z))$ when z is not free in C. We obtain:

$$\equiv\ \exists x\,\forall y\,\exists u\,\forall z\,\Big(P(x)\ \wedge\ \big(\neg Q(x,y)\ \vee\ (R(y,u)\wedge\neg S(u,z))\big)\Big),$$

which is a PNF with a quantifier-free matrix in NNF.

6. **Quantifier entailments: validity and countermodels.** Let R be a binary predicate symbol.

   (**a**) Show that the sequent $\forall x\,\exists y\,R(x,y)\ \vdash\ \exists y\,\forall x\,R(x,y)$ is *not* valid by giving a concrete countermodel with a two-element domain and an explicit interpretation of R.

   (**b**) Prove that the sequent $\exists x\,\forall y\,R(x,y)\ \vdash\ \forall y\,\exists x\,R(x,y)$ *is* valid by a short formal derivation or a clear semantic argument.

Solution. (a) Take domain $\{0,1\}$. Interpret R as the set $\{(0,0),(1,1)\}$. Then for each x we may choose $y=x$, so $\forall x\,\exists y\,R(x,y)$ holds. But there is no single y such that $R(0,y)$ and $R(1,y)$ both hold, so $\exists y\,\forall x\,R(x,y)$ fails. Hence the sequent is not valid.

(b) Valid. From $\exists x\,\forall y\,R(x,y)$ pick a witness a with $\forall y\,R(a,y)$. Let y be arbitrary; taking $x:=a$ yields $R(x,y)$. Since y was arbitrary, $\forall y\,\exists x\,R(x,y)$. Hence the sequent is valid.

Section E. Intro to Cardinality and Diagonalization

Bruno: A central argument which we will see again and again is diagonalization. It was invented by George Cantor at the end of the 19th century. One crucial difficulty in the P vs. NP problem is that it cannot be solved via diagonalization (we may see why when we look at the relativization barrier).

If a set S is finite, it's cardinality $|S|$ is simply the number of elements it contains. But what if S is infinite? How can we talk about cardinality? Cantor realized that the natural way to do it is by grouping things together. He explained this as follows: if one looks at a large cavalry unit in the distance, one might not know how many horses there are and how many men there

are, because they are many and far away and we can't count them all, but we nonetheless know that there are exactly as many men as there are horses. Generalizing this to the infinite case, we say two sets M and H ("men" and "horses") have the same cardinality, which we write $|M| = |H|$, if there exists a bijective function $f : M \to H$. We can then think of $|M|$ has the equivalence class induced by the equivalence relation $|\cdot| = |\cdot|$.

We can also write $|M| \leq |H|$ if there exists an injective function from M to H. The result that $|M| = |H|$ iff $|M| \leq |H|$ and $|H| \leq |M|$ is not obvious - it is known as the Schröder–Bernstein Theorem, and took 10 years after it was first stated by Cantor before a completely formal proof was first found by Berstein. A simple proof of it appears on wikipedia.

1. **Uncountability of infinite $0/1$ sequences.** Let $\{0,1\}^{\mathbb{N}}$ be the set of infinite binary sequences. Suppose $f : \mathbb{N} \to \{0,1\}^{\mathbb{N}}$ is any purported list $f(0), f(1), \ldots$ of such sequences. Show that there exists a sequence which is not in this list. (Hint: flip the i-th bit of the i-th listed sequence). When a set cannot be listed in such fashion, we say it is *uncountable*.

   Solution. The sequence $d_i = 1 - f(i)_i$ is in $\{0,1\}^{\mathbb{N}}$. Nonetheless, for each i, the sequences d and $f(i)$ differ at position i, hence $d \neq f(i)$. Therefore d is not in the image of f, so f cannot be surjective.

2. **Cantor's theorem and uncountability of $(0,1)$.** Let X be any set and let $\mathcal{P}(X)$ be its power set.

   (**a**) Prove *Cantor's theorem*: $|X| < |\mathcal{P}(X)|$, i.e., there is no surjection $f : X \to \mathcal{P}(X)$.

   (**b**) Deduce that there is no surjection $\mathbb{N} \to \mathcal{P}(\mathbb{N})$. Explain how identifying subsets of $\mathbb{N}$ with their characteristic sequences embeds $\mathcal{P}(\mathbb{N})$ into $\{0,1\}^{\mathbb{N}}$, recovering the previous exercise.

   (**c**) Show that the real interval $(0,1)$ is uncountable.

   Solution. (a) Suppose $f : X \to \mathcal{P}(X)$. Let $D = \{x \in X \mid x \notin f(x)\}$. If $D = f(a)$ for some $a \in X$, then $a \in D \iff a \notin f(a) = D$, a contradiction. Hence D is not in the image; f is not surjective.

   (b) Apply (a) to $X = \mathbb{N}$ to conclude no surjection $\mathbb{N} \to \mathcal{P}(\mathbb{N})$. The map $A \subseteq \mathbb{N} \mapsto \chi_A \in \Sigma^{\mathbb{N}}$ is an injection, so $\mathcal{P}(\mathbb{N})$ injects into $\Sigma^{\mathbb{N}}$; uncountability of either implies the other.

   (c) Represent each $x \in (0,1)$ by a binary sequence not eventually all 1's; this makes the coding injective. Given any list of such sequences, flip the i-th bit of the i-th sequence to obtain a new sequence that is not eventually all 1's and hence corresponds to some $y \in (0,1)$ not on the list. Therefore $(0,1)$ is uncountable.